

Sistemas de Información para la Gestión - ICN292

Laboratorio N.º 3

Automatización del triage de devoluciones en n8n

Estudiante: Wan Lo Xie

Rol: 202360654-5

TUT: 21552280-6

Semilla: $S = 280$

Parámetros: $U = 60\,000$, $D = 7$

Fecha: 20 de septiembre de 2026

Repositorio público: <https://github.com/Wan-Lo-Xie/ICN292-Lab3-Xie-Wan-Lo>

Resumen ejecutivo

Se implementó en n8n un producto mínimo viable para automatizar el triage de solicitudes de devolución de AndesHogar SpA. El desarrollo quedó dividido en tres workflows: un flujo principal de triage, un emisor para enviar solicitudes por HTTP POST y un flujo programado para generar el resumen diario. La clasificación utiliza la semilla personal $S = 280$, con umbral de monto $U = 60\,000$ y plazo máximo $D = 7$ días.

El flujo principal recibe cada solicitud mediante un Webhook POST y la evalúa con un nodo Switch en modo Rules. Las reglas se ejecutan con precedencia: primero se revisan datos inválidos, luego rechazo, después revisión y finalmente aprobación. Cada ruta genera un motivo, consulta la API pública de `mindicador.cl` para incorporar el valor de la UF, registra el resultado en una Data Table, envía una notificación de prueba por Gmail y responde al Webhook con la clasificación obtenida. El workflow exportado mantiene la implementación sin nodo Code.

Las 15 solicitudes del archivo entregado fueron clasificadas con los parámetros definidos. El resultado fue de 4 aprobaciones, 4 solicitudes a revisión y 7 rechazos, sin solicitudes inválidas dentro del conjunto oficial. El monto total procesado fue de \$1.251.820 y la tasa de aprobación automática fue de 26,67%. El flujo de resumen consolida estos datos diariamente y genera un único correo con los totales por ruta y monto.

También se probaron errores y casos borde. Un identificador vacío y un monto igual a cero fueron dirigidos correctamente a la ruta `INVALIDA`; en cambio, un monto escrito como "veinte mil" provocó un error de conversión numérica, lo que permite comprobar el comportamiento del flujo ante una entrada no compatible con el formato esperado. Las pruebas de frontera confirmaron que `monto = U` y `dias_desde_compra = D` permanecen dentro de la política. Además, al cambiar intencionalmente la comparación del monto a tipo String, una solicitud de \$9.990 quedó mal enviada a revisión, mostrando por qué la comparación debe ser numérica.

Como limitaciones, el flujo actual no valida la existencia del SKU contra un catálogo y la notificación de triage se probó con una cuenta propia en vez de enviarse directamente al correo lógico del cliente. Para una versión definitiva convendría agregar validación de esquema, consulta de catálogo, manejo centralizado de errores y destinatario dinámico en la notificación.

1. Parámetros y reglas aplicadas

Para la semilla entregada, los parámetros quedan:

$$U = 30\,000 + 1\,000 (280 \bmod 50) = 60\,000$$

$$D = 7 + 7 (280 \bmod 4) = 7$$

La evaluación se realiza en el orden indicado por la política. La primera condición verdadera determina la ruta y el motivo.

Orden	Condición	Ruta	Motivo usado
1	id_solicitud vacío	INVALIDA	ID_SOLICITUD_VACIO
2	monto ≤ 0	INVALIDA	MONTO_NO_MAYOR_QUE_CERO
3	dias_desde_compra $> D$	RECHAZO	PLAZO_MAYOR_A_D
4	estado_producto = danado_por_uso	RECHAZO	DANADO_POR_USO
5	monto $> U$	REVISION	MONTO_MAYOR_A_U
6	estado_producto = con_fallas	REVISION	PRODUCTO_CON_FALLAS
Fallback	Todo lo demás	APROBACION	CUMPLE_POLITICA

Cuadro 1: Reglas implementadas en el nodo Switch.

2. Parte A - Flujo de triage

Estructura del workflow

El workflow ICN292 Lab3 - Triage S280 contiene 13 nodos y no utiliza un nodo Code. El Webhook recibe solicitudes por POST en la ruta lab3-devoluciones-S280. El Switch contiene seis reglas independientes y un fallback de aprobación. Las dos primeras salidas llegan al mismo nodo de datos inválidos, las siguientes dos a rechazo, las siguientes dos a revisión y el fallback a aprobación. De esta forma se conserva la precedencia sin duplicar la lógica posterior.

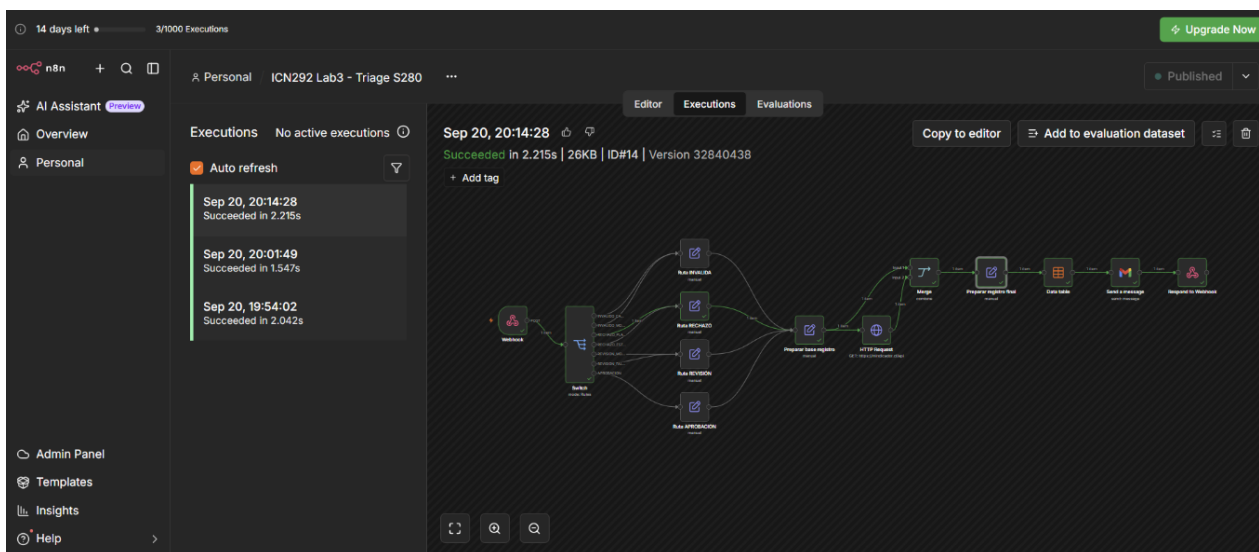


Figura 1: Ejecución exitosa del flujo principal de triage.

Las comparaciones de monto y días utilizan conversión numérica con `toNumber()`, mientras que los estados se comparan como texto. El fallback queda renombrado como APROBACION. La Figura 2

muestra un caso con monto de \$69.980 y cinco días desde la compra, que es enviado a REVISION_MONTO por superar U .

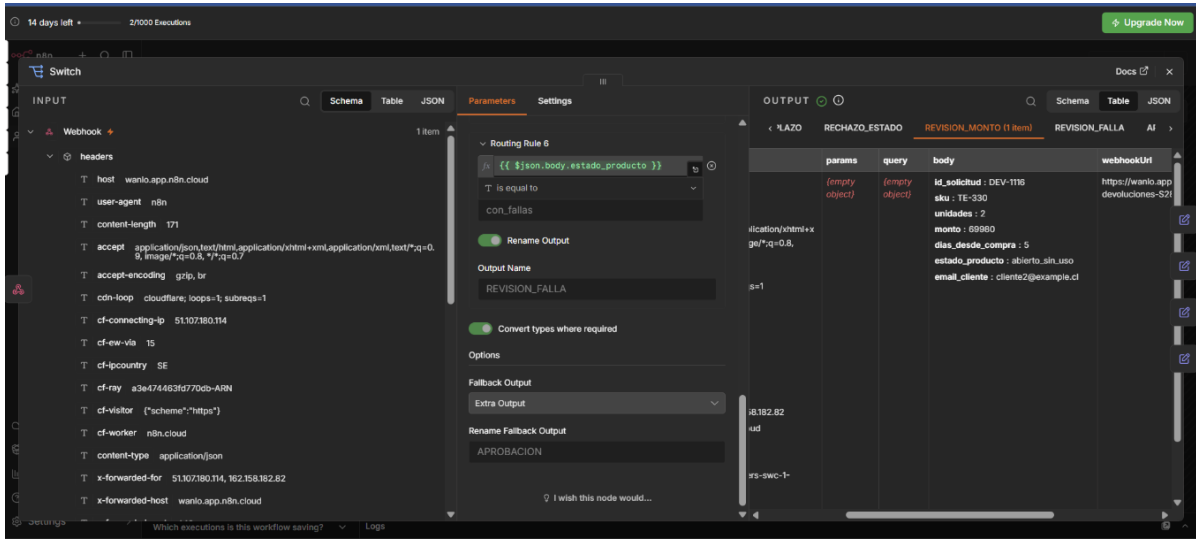


Figura 2: Ejemplo de evaluación del Switch para la solicitud DEV-1116.

Después de asignar *ruta* y *motivo*, se prepara una base de registro y se consulta <https://mindicador.cl/api>. De la respuesta se utiliza el valor de la UF para registrar *uf_valor* y calcular *monto_uf*. Luego se guarda una fila en la Data Table *lab3_registro_devoluciones_S280* con fecha, identificador, SKU, monto, días, estado, correo, ruta, motivo, U , D y datos de UF.

La notificación se implementó con Gmail. Para las pruebas se utilizó una cuenta propia como destinatario, mientras que el correo recibido en la solicitud se conserva dentro del mensaje como “Cliente lógico”. Finalmente, *Respond to Webhook* devuelve un JSON con *id_solicitud*, *ruta*, *motivo*, U y D .

URL de prueba y URL de producción

n8n entrega dos direcciones para un Webhook. La URL de prueba se registra al ejecutar o escuchar un evento desde el editor y permite ver la entrada directamente durante la prueba. La URL de producción se registra cuando el workflow está publicado y sus ejecuciones se revisan desde la pestaña *Executions*. Esta diferencia coincide con la documentación oficial de n8n.¹

En las evidencias y en el workflow emisor se utilizó la URL de producción:

<https://wanlo.app.n8n.cloud/webhook/lab3-devoluciones-S280>

Resultados de las 15 solicitudes

La tabla siguiente resume la clasificación obtenida al aplicar las reglas con $U = 60\,000$ y $D = 7$.

¹n8n, documentación del nodo Webhook: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/>

ID	Monto	Días	Estado	Ruta	Motivo
DEV-1115	19.990	3	sin_abrir	APROBACION	CUMPLE_POLITICA
DEV-1116	69.980	5	abierto_sin_uso	REVISION	MONTO_MAYOR_A_U
DEV-1117	24.990	12	con_fallas	RECHAZO	PLAZO_MAYOR_A_D
DEV-1118	79.990	4	sin_abrir	REVISION	MONTO_MAYOR_A_U
DEV-1119	45.990	25	danado_por_uso	RECHAZO	PLAZO_MAYOR_A_D
DEV-1120	89.990	2	con_fallas	REVISION	MONTO_MAYOR_A_U
DEV-1121	289.990	9	sin_abrir	RECHAZO	PLAZO_MAYOR_A_D
DEV-1122	45.990	2	sin_abrir	APROBACION	CUMPLE_POLITICA
DEV-1123	39.980	15	danado_por_uso	RECHAZO	PLAZO_MAYOR_A_D
DEV-1124	49.980	1	abierto_sin_uso	APROBACION	CUMPLE_POLITICA
DEV-1125	34.990	22	sin_abrir	RECHAZO	PLAZO_MAYOR_A_D
DEV-1126	79.990	8	con_fallas	RECHAZO	PLAZO_MAYOR_A_D
DEV-C01	9.990	1	sin_abrir	APROBACION	CUMPLE_POLITICA
DEV-C02	289.990	2	sin_abrir	REVISION	MONTO_MAYOR_A_U
DEV-C03	79.990	29	abierto_sin_uso	RECHAZO	PLAZO_MAYOR_A_D

Cuadro 2: Resultados de las 15 solicitudes entregadas.

La precedencia se observa, por ejemplo, en DEV-1117 y DEV-1126: aunque ambas solicitudes tienen `con_fallas`, primero cumplen la condición `dias_desde_compra > D`, por lo que terminan en rechazo. Lo mismo ocurre con solicitudes de monto alto pero fuera de plazo, como DEV-1121.

3. Parte B - Flujo programado de resumen

El workflow ICN292 Lab3 - Resumen S280 se programó con *Schedule Trigger* para ejecutarse diariamente a las 20:00. Primero consulta en la Data Table los registros cuya `fecha_dia` coincide con la fecha actual. Después crea indicadores auxiliares por ruta, utiliza *Summarize* para sumar cantidades y montos, arma un solo texto y lo envía mediante Gmail.

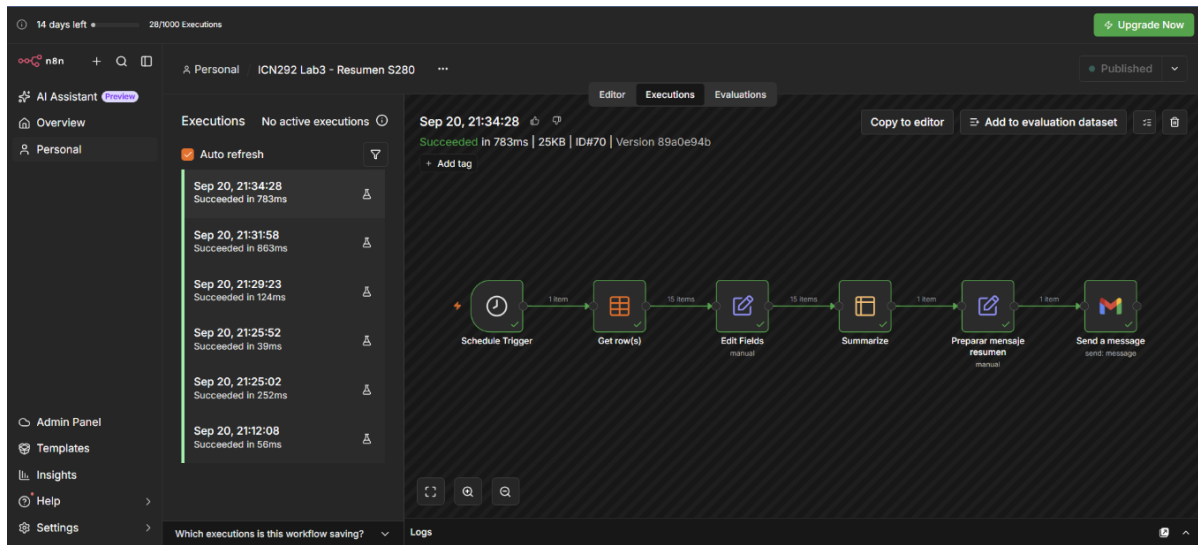


Figura 3: Ejecución del workflow de resumen con 15 registros del día.

La ejecución mostrada consolidó 15 solicitudes: 4 aprobaciones por \$125.950, 4 revisiones por \$529.950 y 7 rechazos por \$595.920. No hubo solicitudes inválidas. El monto total fue de \$1.251.820 y la tasa de aprobación automática fue de 26,67 %.

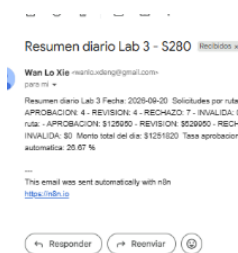


Figura 4: Correo generado por el resumen diario.

Si un día no existen solicitudes, el nodo que consulta la Data Table entrega cero elementos. Con la configuración actual, los nodos posteriores no reciben datos y, por lo tanto, no se genera el correo de resumen. Esto no necesariamente produce un error de ejecución: es un caso en que el flujo puede quedar sin salida visible aunque el disparador sí se haya ejecutado.

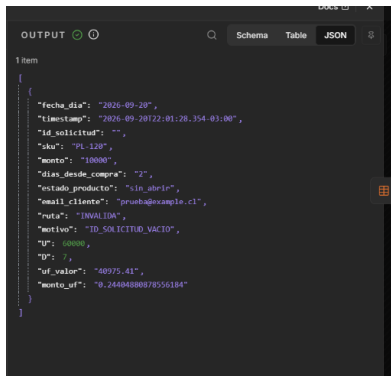
4. Parte C - Errores y casos borde

4.1. Tres entradas inválidas

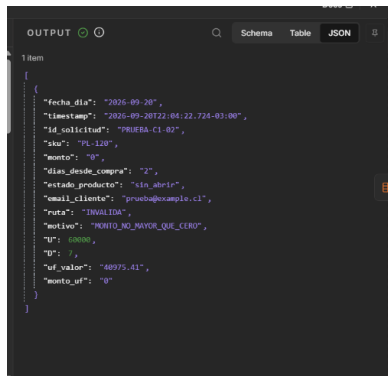
Se probaron tres entradas distintas para observar tanto la ruta de datos inválidos como un fallo real de nodo.

Caso	Carga enviada	Esperado	Observado
C1.1	id_solicitud = , monto = 10.000	Ruta INVALIDA	INVALIDA con motivo ID_SOLICITUD_VACIO.
C1.2	ID válido, monto = 0	Ruta INVALIDA	INVALIDA con motivo MONTO_NO_MAYOR_QUE_CERO.
C1.3	ID válido, monto = "veinte mil"	Que falle al menos un nodo	El Switch no pudo convertir el monto a número y la ejecución se detuvo.

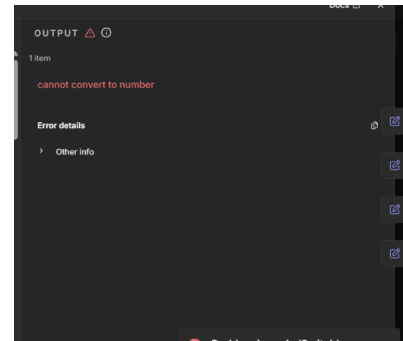
Cuadro 3: Pruebas de entradas inválidas.



(a) ID vacío.



(b) Monto igual a cero.



(c) Error de conversión numérica.

Figura 5: Evidencia de los tres casos inválidos.

4.2. On Error y Error Workflow

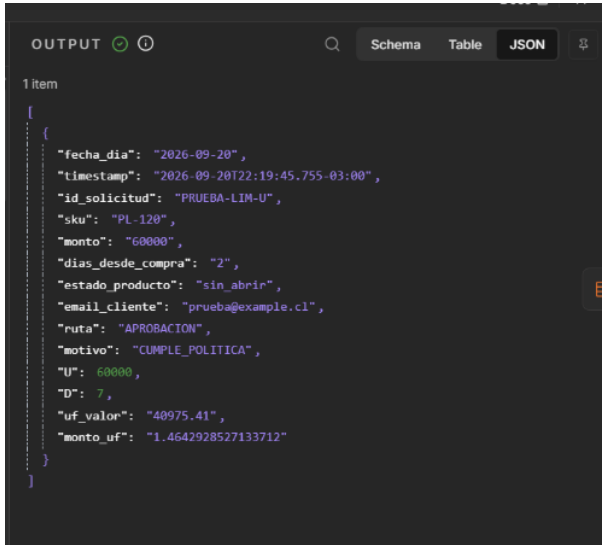
En los nodos de n8n se dispone de tres comportamientos generales ante error: *Stop Workflow*, *Continue* y *Continue (using error output)*. En los workflows exportados no se configuró un valor distinto del comportamiento por defecto, por lo que se mantuvo *Stop Workflow*. La razón es evitar que una solicitud siga avanzando después de una conversión, consulta, registro o notificación fallida y termine guardándose como si el proceso hubiera sido correcto.

En particular, se mantuvo este criterio en el Switch, HTTP Request, Data Table, Gmail, Summarize y Respond to Webhook. Los nodos Set y los disparadores también conservan la configuración por defecto, ya que no se necesitaba una rama alternativa de recuperación para este laboratorio.

Un *Error Workflow* es un flujo separado destinado a reaccionar frente al error de otro workflow, por ejemplo registrando el fallo o enviando una alerta. Es útil para errores reales de ejecución. Sin embargo, no soluciona por sí solo un día sin solicitudes: si el Schedule Trigger se ejecuta, la consulta devuelve cero elementos y el resto no corre, puede no existir un error técnico que dispare ese mecanismo. Por eso un proceso programado puede dejar de producir su salida esperada sin que aparezca como una falla explícita.

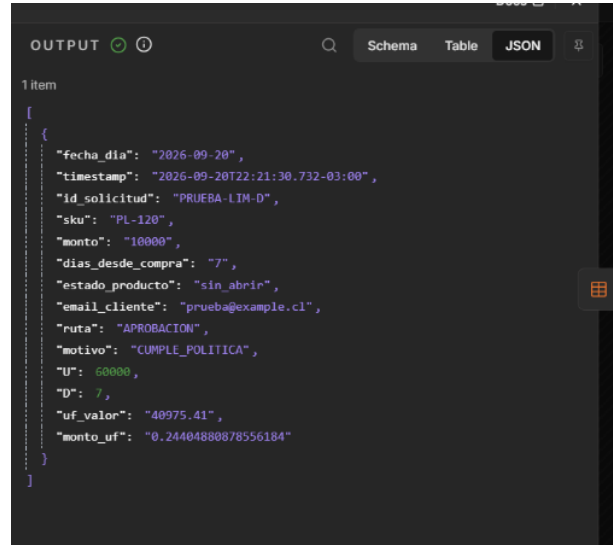
4.3. Límites U y D

Las comparaciones son inclusivas respecto del límite permitido: el rechazo usa `días >D` y la revisión por monto usa `monto >U`. Por lo tanto, un monto exactamente igual a U no se manda a revisión y un plazo exactamente igual a D sigue dentro de plazo.



```
{
  "fecha_dia": "2026-09-20",
  "timestamp": "2026-09-20T22:19:45.755-03:00",
  "id_solicitud": "PRUEBA-LIM-U",
  "sku": "PL-120",
  "monto": "60000",
  "dias_desde_compra": "2",
  "estado_producto": "sin_abrir",
  "email_cliente": "prueba@example.cl",
  "ruta": "APROBACION",
  "motivo": "CUMPLE_POLITICA",
  "U": "60000",
  "D": "7",
  "uf_valor": "40975.41",
  "monto_uf": "1.4642928527133712"
}
```

(a) Monto exactamente igual a $U = 60\,000$: APROBACION.



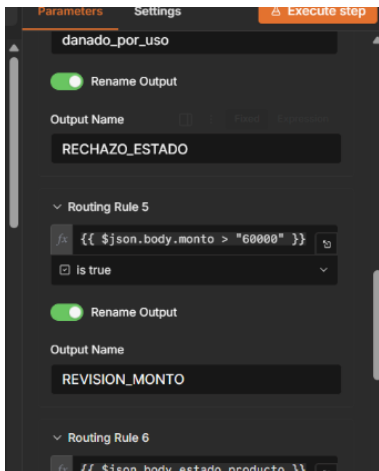
```
{
  "fecha_dia": "2026-09-20",
  "timestamp": "2026-09-20T22:21:30.732-03:00",
  "id_solicitud": "PRUEBA-LIM-D",
  "sku": "PL-120",
  "monto": "10000",
  "dias_desde_compra": "7",
  "estado_producto": "sin_abrir",
  "email_cliente": "prueba@example.cl",
  "ruta": "APROBACION",
  "motivo": "CUMPLE_POLITICA",
  "U": "60000",
  "D": "7",
  "uf_valor": "40975.41",
  "monto_uf": "0.24404880878556184"
}
```

(b) Días exactamente iguales a $D = 7$: APROBACION.

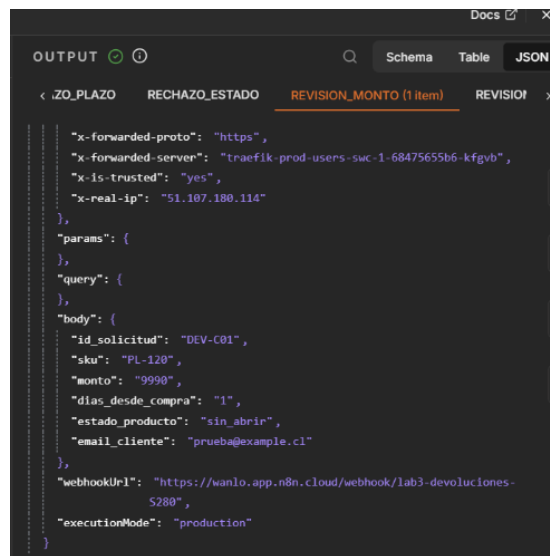
Figura 6: Pruebas construidas para los límites de la política.

4.4. Error de tipo provocado

Para comprobar el efecto del tipo de dato se modificó temporalmente la condición de monto, pasando de una comparación numérica a una comparación de texto. Se envió DEV-C01, cuyo monto es \$9.990. Numéricamente, $9\,990 < 60\,000$ y debería aprobarse, pero como texto la comparación comienza por el primer carácter y "9" queda por encima de "6". El resultado fue una clasificación incorrecta en REVISION_MONTO. Después de la prueba se restauró la regla numérica del workflow final.



(a) Regla temporal usando comparación String.



```
{
  "x-forwarded-proto": "https",
  "x-forwarded-server": "traefik-prod-users-swc-1-68475655b6-kfgvb",
  "x-is-trusted": "yes",
  "x-real-ip": "51.107.180.114",
  "params": {
  },
  "query": {
  },
  "body": {
    "id_solicitud": "DEV-C01",
    "sku": "PL-120",
    "monto": "9990",
    "dias_desde_compra": "1",
    "estado_producto": "sin_abrir",
    "email_cliente": "prueba@example.cl"
  },
  "webhookUrl": "https://wanlo.app.n8n.cloud/webhook/lab3-devoluciones-S280",
  "executionMode": "production"
}
```

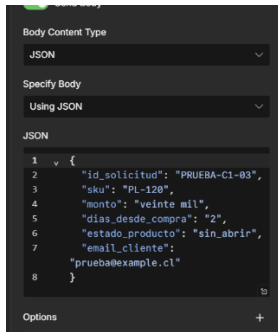
(b) DEV-C01 enviado erróneamente a revisión.

Figura 7: Efecto del cambio intencional de tipo de dato.

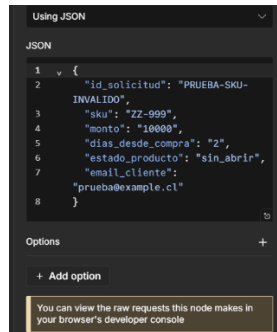
4.5. Dos casos que el flujo no puede resolver

El primer caso es un monto expresado como texto natural, por ejemplo "veinte mil". La lógica necesita convertir el monto a número antes de compararlo y no existe una regla previa que valide este formato. Por eso la conversión falla. Para detectarlo de forma controlada se necesitaría una validación de esquema o de tipo antes del Switch de negocio, separando formato inválido de las reglas de clasificación.

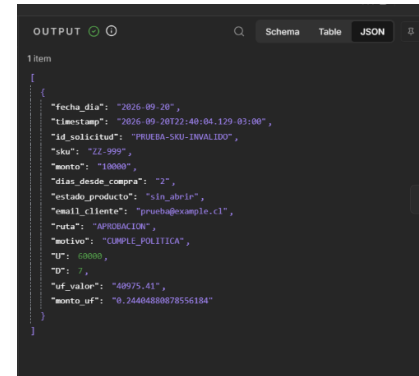
El segundo caso es un SKU inexistente, por ejemplo ZZ-999. El workflow actual no consulta el catálogo de productos; solo evalúa identificador, monto, días y estado. Por esa razón, si los demás campos parecen válidos, el SKU desconocido puede terminar aprobado. Para resolverlo se necesitaría consultar una fuente de catálogo y agregar una regla que rechace o derive a revisión los SKU no encontrados.



(a) Monto escrito como texto.



(b) Entrada con SKU ZZ-999.



(c) El SKU inexistente queda aprobado.

Figura 8: Casos que requieren validaciones adicionales al flujo actual.

5. Conclusiones

El desarrollo permite automatizar la recepción, clasificación, registro y resumen de las devoluciones usando la lógica solicitada para $S = 280$, sin reemplazar el Switch por código. Las pruebas muestran que la precedencia funciona, que los límites U y D se tratan correctamente y que el resumen diario coincide con las 15 solicitudes procesadas.

Las principales mejoras para convertir el MVP en una solución definitiva serían validar tipos antes del triage, comprobar el SKU contra un catálogo, usar el correo del cliente como destinatario real y agregar manejo centralizado de errores. El bonus con nodo Code no fue implementado, ya que la entrega principal se mantuvo con Switch Rules, tal como se solicita en la Parte A.